

Assignment 5 - Part 2

Data Wrangling with R

Team Number	2
Project Title	Historical Stock Analysis
Dataset	data/stock_data.csv - https://www.kaggle.com/datasets/ibrahimshahrukh/top-50-companies-dataset (the source page has since been removed by its uploader)
Team Members	Karthikeyan M (26120004), Mirthula M (26120121), Lakshmi Mayuri Kavya N (26120161)
Date	19 August 2026

The same stock data as in Assignment 4, but with 230,111 rows. In this part we converted wide and long data using tidyr, used data.table to manipulate the Date column, and used dplyr and data.table to get the same grouped summary in two different ways. The stocks are in two different currencies, so we don't compare prices directly, we compare indexed prices or returns instead.

B.1 Reshape your data

We took the last trading day in each month for 2025, using the following stocks: AAPL, MSFT, NVDA, AVGO. In January we then set our base figure at 100 for all of the stocks. All four stocks are traded in the U.S. and can be indexed with a different starting price, but the change over time is similar.

Wide and long forms

Table 1. Wide form (first four months; January 2025 = 100)

Month	AAPL	AVGO	MSFT	NVDA
Jan 2025	100.0	100.0	100.0	100.0
Feb 2025	102.6	90.1	95.8	104.0
Mar 2025	94.2	75.9	90.6	90.3
Apr 2025	90.1	87.2	95.4	90.7

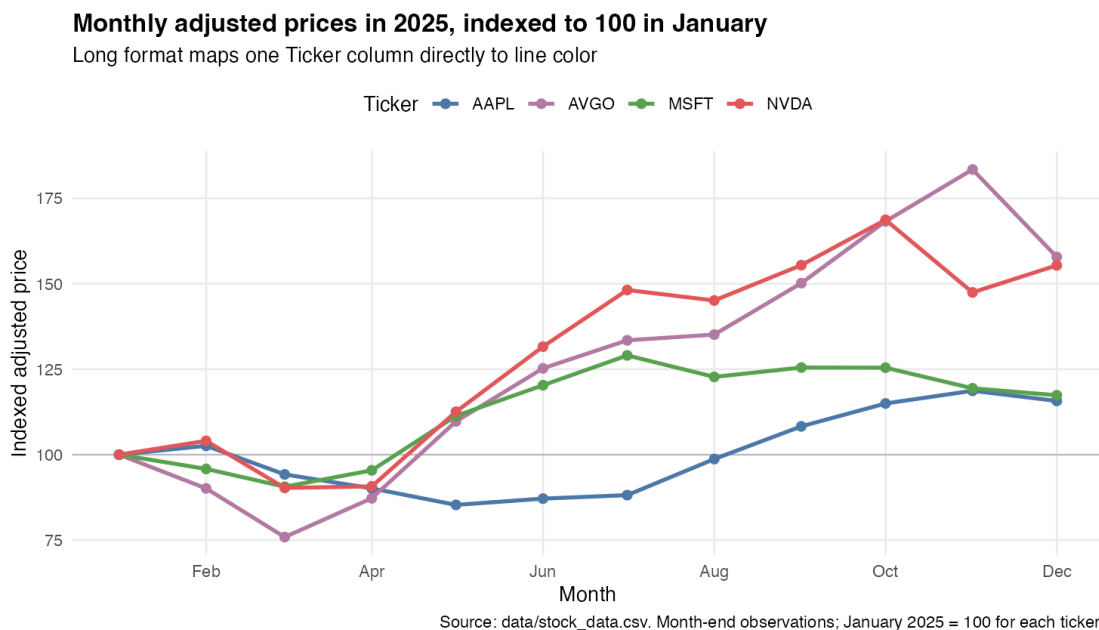
Table 2. Long form (first eight rows)

Month	Ticker	Indexed Price
Jan 2025	AAPL	100.0
Jan 2025	AVGO	100.0
Jan 2025	MSFT	100.0
Jan 2025	NVDA	100.0
Feb 2025	AAPL	102.6
Feb 2025	AVGO	90.1
Feb 2025	MSFT	95.8
Feb 2025	NVDA	104.0

```
monthly_wide <- monthly_long_source %>%  
  pivot_wider(names_from = Ticker, values_from = Indexed.Price)
```

```
monthly_long <- monthly_wide %>%  
  pivot_longer(cols = all_of(selected_tickers),  
    names_to = "Ticker", values_to = "Indexed.Price")
```

Figure 1. Monthly adjusted prices in 2025, indexed to 100 in January.



The plot for the long form was easier to plot in ggplot2, which could be used to color the line and add all four stocks to a single `geom_line()` layer. In the wide form, each ticker will be a separate column and this will be helpful for reading the table but not so handy for this chart. It is also obvious in Figure 1 that AVGO and NVDA have ended up at 2025 and 155 respectively while AAPL and MSFT have finished at 116 and 117 respectively.

B.2 Parse and use a date/time field

As in Assignment 4, the Date column entered into R was plain text. We used `ymd()` to take out the year, month, and weekday data, and `floor_date()` to group the daily rows by month.

Table 3. Parsed date fields (sample)

Date	Ticker	Year	Month	Weekday
2006-01-02	000660.KS	2006	January	Monday
2006-01-03	000660.KS	2006	January	Tuesday
2006-01-04	000660.KS	2006	January	Wednesday

Table 4. Five months with the highest mean absolute daily return

Month	Mean Absolute Return	Tickers
October 2008	4.82%	40
March 2020	4.67%	48
November 2008	4.38%	40
March 2009	3.25%	41
January 2009	3.16%	40

```
stocks <- stocks %>%
  mutate(Date = lubridate::ymd(Date),
         Year = lubridate::year(Date),
         Month.Name = lubridate::month(Date, label = TRUE),
         Weekday = lubridate::wday(Date, label = TRUE),
         Month = lubridate::floor_date(Date, "month"))

monthly_volatility <- stocks %>%
  filter(!is.na(Daily.Return)) %>%
  group_by(Ticker, Month) %>%
  summarise(Mean.Abs.Return = mean(abs(Daily.Return)), .groups = "drop") %>%
  group_by(Month) %>%
  summarise(Mean.Abs.Return = mean(Mean.Abs.Return),
           Tickers = n(), .groups = "drop")
```

Figure 2. Average absolute daily return by month, equal-weighted across available tickers.

When the dates are listed by month, the periods of high volatility are much more apparent. October 2008 is the highest month at 4.82%, followed by March 2020 at 4.67% and November 2008 at 4.38%. First we averaged across all the tickers' interiors within each ticker-month, and then we averaged across the ticker-months. This prevents one market with more companies driving the results by itself.

B.3 data.table vs. dplyr

We repeated Assignment 4 C3.1, grouping daily returns by market, and calculated the mean return, the standard deviation of the returns, and the number of ticker-day observations.

Code comparison. The same grouped summary in two syntaxes

dplyr	data.table
<pre>market_stats_dplyr <- returns_by_market %>% group_by(Market) %>% summarise(mean_return = mean(Daily.Return), sd_return = sd(Daily.Return), observations = n(), .groups = "drop") %>% arrange(desc(mean_return))</pre>	<pre>market_stats_dt <- returns_dt[, .(mean_return = mean(Daily.Return), sd_return = sd(Daily.Return), observations = .N), by = Market][order(-mean_return)]</pre>

Table 5. Result from both dplyr and data.table

Market	Mean Return	SD Return	Observations
Korea	0.094%	2.37%	9,942
United States	0.086%	2.19%	184,754
Hong Kong	0.079%	1.99%	18,528
Paris	0.063%	1.82%	5,151
Switzerland	0.036%	1.27%	10,138
Saudi Arabia	0.015%	1.05%	1,549

The results are the same for both versions. We liked dplyr to be easier to read because group_by() and summarise() reveal the steps in the order in which we consider them. The data.table version is short, and we'd hope it would be faster and take up less space on a much larger data set. Both worked here, with 230,111 rows, in which case readability had more importance.

B.4 Reflection

Yes. We were able to make a few stock lines from the same ggplot mapping without reshaping the data, and we could see the exact timing of the big volatility bursts after we grouped a column of the data by month – they were there but the shape and group by month made them apparent.

We used ChatGPT for help with wording and formatting.